AI-901

Microsoft Foundry Implementation Cheat Sheet

Focused on the 55–60% “Implement AI solutions by using Microsoft Foundry” domain

Purpose: A compact, exam-focused reference for recognizing and understanding the Python/API patterns that can reasonably appear on AI-901. It intentionally avoids AI-102-level implementation depth.

Blueprint basis: skills measured as of April 15, 2026 • Microsoft study guide last updated July 13, 2026

Use this guide to	Do not spend time memorizing
Recognize client objects, endpoints, credentials, methods, and outputs	Every constructor argument, error type, or advanced SDK feature
Know which service solves which workload	AI-102 architecture depth or production hardening
Understand portal → code equivalence	All model-specific options and preview features
Spot common exam distractors	Large complete programs line-by-line

1. Exam blueprint: what the 55–60% actually contains

Official scope: Microsoft states that candidates need conceptual knowledge plus foundational technical skills, Python syntax/programming techniques, familiarity with Azure resources, and familiarity with REST APIs, SDKs, and CLIs.

Domain	What Microsoft explicitly lists
Generative AI apps & agents	System/user prompts; deploy/interact with model; lightweight chat client with Foundry SDK; create/test single agent; lightweight agent client
Text & speech	Lightweight text-analysis app; respond to spoken prompts with a deployed multimodal model; lightweight Azure Speech app
Vision & image generation	Interpret visual prompt input; create visual outputs; lightweight app with vision capabilities
Information extraction	Documents/forms, images, audio/video with Content Understanding; lightweight extraction app

The repeated implementation pattern

```
1. Get endpoint / deployment / resource details
2. Authenticate (key or Microsoft Entra ID)
3. Create a client object
4. Call the correct operation / method
5. Read the returned result
```

EXAM RULE: If you can identify those five pieces in a snippet, you can answer a large percentage of implementation questions without memorizing the entire SDK.

Must-recognize vocabulary

Term	Meaning for AI-901
Foundry project	Workspace context for models, agents, connections, tools, data, and related assets.
Deployment name	The name of the deployed model instance passed to API calls; it is not always identical to the base model name.
Endpoint	Service URL used by the client to send requests.
Credential / API key	Proof that the client is allowed to call the resource.
Lightweight client	Small CLI/simple app that collects input, calls a remote AI service, and displays results.
Responses API	Current OpenAI-compatible API pattern used in current Foundry learning content.
Tool	Callable capability used by an agent to act.
Knowledge	External content retrieved to ground an agent response.
Analyzer + schema	Content Understanding: analyzer applies a schema to input and returns structured results.

2. Foundry models: prompts, deployments, Responses API

System vs user prompt

Prompt type	Purpose	Exam cue
System / instructions	Defines role, behavior, style, constraints, output rules, and guardrails.	“You are a concise tutor…”, “Always return JSON…”, “Do not…”
User	The actual end-user request or task.	“Summarize this…”, “What is in this image?”

Canonical model-call pattern

```
from openai import OpenAI

client = OpenAI(
    base_url=endpoint,
    api_key=api_key
)

response = client.responses.create(
    model=deployment_name,
    input=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Summarize this text."}
    ],
    max_output_tokens=300,
    temperature=0.7
)

print(response.output_text)
```

RECOGNIZE: `OpenAI(...)` creates the client; `responses.create(...)` invokes the deployed model; `model=` expects the deployment name; `response.output_text` is a convenience property for generated text.

Runtime parameters worth knowing

Parameter	What it changes	Likely distractor
temperature	Creativity/randomness vs determinism	It does not set response length.
max_output_tokens	Caps generated output length / token usage	It does not choose the model.
instructions / system message	Behavior and role	It is not the user’s task.
model	Selects the deployed model by deployment name	Do not confuse with endpoint.

Portal → code: The Foundry Playground is used to test prompts, models, system instructions, and parameters. The same settings can then be replicated in application code.

3. Agents: model + instructions + tools

Mental model: Model = raw intelligence. Agent = packaged task-oriented behavior built on a model, with instructions and optionally tools/knowledge.

Agent component	Meaning
Model	The generative model used for reasoning.
Instructions	System-level behavior: role, style, constraints, output rules.
Tools	Actions the agent can call: functions/APIs, Code Interpreter, MCP tools, retrieval, etc.
Knowledge	External content used for grounded answers; often supports citations.

Agent client pattern from the current Microsoft Learn module

```
from azure.identity import DefaultAzureCredential
from azure.ai.projects import AIProjectClient

project_client = AIProjectClient(
    endpoint=project_endpoint,
    credential=DefaultAzureCredential(),
)

agent = project_client.agents.get(agent_name="learning-agent")
openai_client = project_client.get_openai_client()

response = openai_client.responses.create(
    input=[{"role": "user", "content": "Tell me what you can help with."}],
    extra_body={"agent": {"name": agent.name, "type": "agent_reference"}},
)
print(response.output_text)
```

RECOGNIZE: `DefaultAzureCredential()` = Microsoft Entra-based credential chain; `AIProjectClient(...)` = Foundry Project client; `.agents.get(...)` retrieves an existing agent; `.get_openai_client()` creates an authenticated OpenAI-compatible client; `agent_reference` tells the Responses API to use the agent.

MCP in one exam-ready paragraph

Model Context Protocol (MCP) is an open client-server standard for exposing tools and data to AI agents. The agent/host acts as the MCP client; an MCP server exposes callable tools/data. In current Microsoft Learn content, Azure Language can be connected to an agent through an Azure Language MCP server, giving deterministic text-analysis capabilities without hand-coding every REST call.

Common trap: A plain model can generate answers. An agent adds reusable behavior and can use tools/knowledge to act or ground its answers.

4. Text analysis: generative model vs Azure Language

Use a general-purpose model when…	Use Azure Language when…
You want flexible conversational analysis	You need structured, predictable outputs
A prompt can describe the task	You need a purpose-built operation such as language or PII detection
Variation in wording is acceptable	You need confidence scores / typed entities / redacted text

Azure Language SDK - language detection

```
from azure.core.credentials import AzureKeyCredential
from azure.ai.textanalytics import TextAnalyticsClient

client = TextAnalyticsClient(
    endpoint=language_endpoint,
    credential=AzureKeyCredential(key)
)

result = client.detect_language([text])[0]
print(result.primary_language.name)
print(result.primary_language.iso6391_name)
print(result.primary_language.confidence_score)
```

Azure Language SDK - PII detection

```
result = client.recognize_pii_entities([text])[0]
print(result.redacted_text)
for entity in result.entities:
    print(entity.text, entity.category, entity.confidence_score)
```

Operation / concept	Know this
Language detection	Returns detected language, ISO code, confidence score.
PII recognition	Returns redacted text plus detected entities/categories/confidence.
Named entity recognition	Identifies entities such as people, organizations, locations.
Sentiment analysis	Evaluates sentiment/opinion in text.
Key phrase extraction	Returns important phrases/topics.
Summarization	Condenses text; can be handled by a generative model or specialized capability depending on scenario.

EXAM CHOICE: If the question stresses “predictable,” “structured,” “confidence score,” “redaction,” or a specific NLP operation, prefer Azure Language. If it stresses flexible natural-language reasoning, prefer a generative model.

5. Speech: speech-to-text, text-to-speech, and Voice Live

```
pip install azure-cognitiveservices-speech
```

Speech-to-text recognition pattern

```
import azure.cognitiveservices.speech as speechsdk

speech_config = speechsdk.SpeechConfig(
    subscription=speech_key,
    endpoint=endpoint_url
)
audio_config = speechsdk.audio.AudioConfig(use_default_microphone=True)

speech_recognizer = speechsdk.SpeechRecognizer(
    speech_config=speech_config,
    audio_config=audio_config
)

speech_recognizer.start_continuous_recognition()
# ... receive recognizing/recognized events ...
speech_recognizer.stop_continuous_recognition()
```

Text-to-speech synthesis pattern

```
speech_config = speechsdk.SpeechConfig(subscription=key, endpoint=endpoint)
audio_config = speechsdk.audio.AudioOutputConfig(use_default_speaker=True)
speech_config.speech_synthesis_voice_name = "<voice-name>"

synthesizer = speechsdk.SpeechSynthesizer(
    speech_config=speech_config,
    audio_config=audio_config
)
result = synthesizer.speak_text_async(text).get()
```

Object / term	Purpose
SpeechConfig	Authentication + Speech service configuration.
AudioConfig	Input source for recognition, e.g. microphone/file.
AudioOutputConfig	Output destination for synthesized speech.
SpeechRecognizer	Converts speech/audio to text.
SpeechSynthesizer	Converts text to audio.
Voice Live	Managed real-time voice conversation capability combining speech with generative reasoning/agent behavior.

Pipeline to remember: Speech-to-text → reasoning/processing → text-to-speech. Voice Live packages this into a real-time conversational experience and can be enabled for a Foundry agent.

6. Vision and multimodal prompts

Core concept: A multimodal model can process more than one modality in the same request. For AI-901, the important code pattern is text + image in one prompt.

Vision-enabled model call

```
response = client.responses.create(
    model=deployment_name,
    input=[
        {
            "role": "user",
            "content": [
                {"type": "input_text", "text": "What is in this image?"},
                {"type": "input_image", "image_url": image_url}
            ]
        }
    ]
)
print(response.output_text)
```

Input / output pattern	What it means
Text + image → text	Visual question answering, image description, chart/screenshot/document interpretation.
Image by URL	The request references an accessible image URL.
Image as base64 data	Binary image is encoded as text so it can travel in JSON/API payloads.
Multimodal model	One model handles both the text instruction and visual input.

RECOGNIZE: `type: input_text` and `type: input_image` are the key clues that a Responses API request contains multimodal input.

7. Image generation: text → new visual output

Current Microsoft Learn image-generation pattern

```
import base64

response = client.responses.create(
    model=image_deployment,
    input="A modern flat illustration of a robot holding a plant.",
    tools=[{"type": "image_generation"}],
)

image_base64 = next(
    item.result for item in response.output
    if item.type == "image_generation_call"
)

with open("generated.png", "wb") as f:
    f.write(base64.b64decode(image_base64))
```

Question cue	Correct idea
“Describe this image”	Use a multimodal/vision-enabled model.
“Create an image from this description”	Use an image-generation model/capability.
“Save generated image bytes”	Decode returned base64 before writing binary image file.
“Which setting selects the deployed model?”	The deployment/model name passed to `model=`.

Do not overlearn model names: Model availability changes. For AI-901, focus on capability selection and code structure. Current Learn content references the GPT-Image-1 family, but exam logic should be based on workload/capability, not memorizing a vendor catalog.

8. Content Understanding: documents, images, audio, video

High-value 2026 topic: Content Understanding converts unstructured multimodal content into structured data. It uses analyzers and schemas, and can return fields, markdown/transcripts, and confidence information.

Concept	Exam-ready meaning
Schema	Defines which fields/entities to extract and how they are structured; can include nested/collection fields.
Analyzer	Applies AI processing + schema consistently to incoming content and returns structured results.
Prebuilt analyzer	Ready-made analyzer for common scenarios such as invoices, image/audio/video search.
Custom analyzer	Analyzer configured with a custom schema for your business-specific fields.
LRO / asynchronous analysis	Content analysis can take time; SDK returns a poller and waits/polls until completion.
Structured output	Typically JSON-like fields plus extracted content/markdown/transcript.

Content Understanding SDK pattern

```
from azure.ai.contentunderstanding import ContentUnderstandingClient
from azure.core.credentials import AzureKeyCredential

client = ContentUnderstandingClient(
    endpoint=foundry_endpoint,
    credential=AzureKeyCredential(key)
)

analyzer_id = "prebuilt-invoice"
inputs = [{"url": invoice_url}]

poller = client.begin_analyze(
    analyzer_id=analyzer_id,
    inputs=inputs
)
result = poller.result()

for content in result.contents:
    print(content.markdown)
    print(content.fields)
```

RECOGNIZE: `begin_analyze(...)` signals a long-running/asynchronous operation. `poller.result()` waits for completion. `content.fields` is structured extracted data; `content.markdown` can hold extracted textual representation.

Prebuilt analyzer example	Typical use
prebuilt-invoice	Extract invoice-specific structured fields.
prebuilt-imageSearch	Analyze/search image content.

prebuilt-audioSearch	Analyze audio, often with transcript/insights.
prebuilt-videoSearch	Analyze video content/insights.

9. Authentication, endpoints, and deployment names

Pattern	What to infer
`AzureKeyCredential(key)`	API-key authentication for an Azure SDK client.
`DefaultAzureCredential()`	Microsoft Entra ID credential chain; avoids hard-coding a key.
`OpenAI(base_url=..., api_key=...)`	OpenAI-compatible client pointed at a Foundry/Azure endpoint.
`AIProjectClient(endpoint=..., credential=...)`	Foundry project-level client.
Environment variables / `.env`	Recommended way to keep endpoint/key/deployment settings outside source code.
`model=deployment_name`	Select deployed model instance. Deployment name may differ from base model name.

```
# Typical environment-driven configuration
load_dotenv()
endpoint = os.getenv("AZURE_OPENAI_ENDPOINT")
api_key = os.getenv("API_KEY")
deployment_name = os.getenv("MODEL_DEPLOYMENT_NAME")
```

Security instinct: If an exam option hard-codes production secrets directly into source code and another option uses a credential mechanism/environment configuration, the latter is usually the better design. AI-901 may test recognition rather than full security architecture.

Do not confuse…	With…
Endpoint	Deployment/model name
API key / credential	Endpoint
Base model name	Your deployment name
Project client	Model response object
System instructions	User input

10. High-probability exam traps and decision rules

Flexible reasoning vs deterministic NLP: Generative model for flexible prompt-driven analysis; Azure Language for predictable structured NLP operations.

Model vs agent: Model = inference. Agent = model + reusable instructions + tools/knowledge.

Vision vs image generation: Vision interprets visual input; image generation creates new visual output.

Speech recognition vs synthesis: Recognition = audio → text. Synthesis = text → audio.

Content Understanding vs OCR: OCR extracts text. Content Understanding can semantically map extracted content into schema-defined structured fields.

Synchronous vs long-running: Content Understanding analysis commonly uses begin_* + poller.result() pattern.

Portal vs SDK: Portal/playground validates models/prompts/settings; SDK/API brings them into your application.

Tool vs knowledge: Tool performs an action/call; knowledge provides context for grounding/retrieval.

API endpoint vs deployment name: Endpoint locates service; deployment name selects model instance.

MCP client vs server: Agent/host = client; MCP server exposes tools/data/actions.

Fast elimination strategy: First identify the workload (model/agent, language, speech, vision, extraction). Then identify the direction of data flow. Finally identify the expected result type (free text, structured fields, transcript, audio, image). This usually eliminates most distractors.

11. Practice questions: code recognition

1. A snippet calls `client.responses.create(model=deployment_name, input=...)`. What identifies the deployed model?

Answer: The value passed to `model`, typically the deployment name.

2. A requirement says output must include language ISO code and a confidence score. Generative model or Azure Language?

Answer: Azure Language; it returns structured purpose-built results.

3. Which object converts microphone audio into recognized text?

Answer: `SpeechRecognizer`.

4. Which object converts text into audible speech?

Answer: `SpeechSynthesizer`.

5. What does `DefaultAzureCredential()` represent?

Answer: Microsoft Entra-based credential chain used by Azure SDK clients.

6. Why use `AIProjectClient`?

Answer: To connect to and work with a Microsoft Foundry project and project-level capabilities, including agents.

7. What does an `agent_reference` indicate in a Responses API call?

Answer: The response should use an existing Foundry agent rather than only a raw model call.

8. A request contains `input_text` plus `input_image`. What workload is this?

Answer: Multimodal/vision analysis.

9. What is the clue that a Content Understanding call is long-running?

Answer: `begin_analyze(...)` returns a poller; `.result()` waits for completion.

10. What is the main difference between OCR and schema-based Content Understanding?

Answer: OCR extracts text; Content Understanding maps content semantically into structured schema-defined fields.

11. What is the purpose of a system prompt/instructions?

Answer: Set role, behavior, constraints, style, and output rules.

12. What does temperature control?

Answer: Randomness/creativity vs determinism.

13. What does `max_output_tokens` control?

Answer: Maximum generated output length/token budget.

14. A user asks for a newly generated illustration. Vision model or image generation?

Answer: Image generation.

15. An agent needs a callable language-analysis service through a standard protocol. What concept fits?

Answer: MCP server/tool integration.

16. Where should you first test prompts and deployment settings before copying into code?

Answer: Foundry Playground.

17. If an API response must be predictable and structured, what service style is usually preferable?

Answer: Purpose-built service such as Azure Language / Content Understanding rather than unrestricted generative text.

18. What is a schema in Content Understanding?

Answer: A definition of the fields/entities and structure you want extracted.

19. What is an analyzer?

Answer: A reusable processing unit that applies analysis/schema and returns structured results.

20. For audio conversation without manually stitching STT + reasoning + TTS, what current Azure Speech capability should you recognize?

Answer: Voice Live.

12. Efficient study plan for this 55–60% domain

Session	Focus	What “done” means
1 • 60 min	Blueprint + Foundry basics	Can explain endpoint, key/credential, deployment name, project, playground.
2 • 75 min	Generative models + Responses API	Can read a model-call snippet and identify system/user prompts and parameters.
3 • 75 min	Agents + AIProjectClient + MCP	Can explain model vs agent, tools vs knowledge, and agent-call flow.
4 • 60 min	Azure Language	Can identify TextAnalyticsClient and choose deterministic NLP vs generative model.
5 • 60 min	Speech	Can distinguish STT/TTS/Voice Live and recognize recognizer/synthesizer objects.
6 • 60 min	Vision + image generation	Can distinguish image understanding from creation and read multimodal request structure.
7 • 75 min	Content Understanding	Can explain analyzer/schema/LRO and read begin_analyze + result + fields.
8 • 45 min	Mixed code recognition	Redo the 20 questions above without looking at answers.
9 • 30–45 min	Microsoft Practice Assessment	Use misses to revisit only the weak domain.

Best use of hands-on labs: Run each official Foundry lab once. You are not trying to memorize UI clicks; you are building recognition of how the portal configuration maps to code and how each workload returns results.

What to memorize vs what to understand

Memorize / instantly recognize	Understand conceptually
`responses.create`, `AIProjectClient`, `DefaultAzureCredential`	Why models and agents differ
`TextAnalyticsClient`, `AzureKeyCredential`	Why purpose-built NLP is more predictable
`SpeechRecognizer`, `SpeechSynthesizer`	Speech direction: audio↔️text
`input_text` + `input_image`	Multimodal reasoning
`ContentUnderstandingClient`, `begin_analyze`, `poller.result`	Schema/analyzer/LRO flow
`model=deployment_name`	Deployment name vs base model name

13. Final-day one-page cram sheet

```
MODEL CALL
OpenAI(...) → responses.create(model=DEPLOYMENT, input=...) → response.output_text

AGENT CALL
DefaultAzureCredential → AIProjectClient → agents.get → get_openai_client → responses.create(...
agent_reference ...)

AZURE LANGUAGE
TextAnalyticsClient(endpoint, AzureKeyCredential(key))
  detect_language(...)
  recognize_pii_entities(...)

SPEECH
SpeechRecognizer = audio → text
SpeechSynthesizer = text → audio
Voice Live = real-time conversational voice

VISION
input_text + input_image → multimodal model → text answer

IMAGE GENERATION
text prompt + image_generation tool/model → base64/image output

CONTENT UNDERSTANDING
ContentUnderstandingClient → begin_analyze(analyzer_id, inputs) → poller.result() →
contents/fields/markdown

DECISION RULES
Flexible reasoning → generative model
Predictable structured NLP → Azure Language
Interpret image → multimodal model
Create image → image generation
Extract schema-defined data from multimodal content → Content Understanding
```

If you only have 30 minutes: Review this page, then sections 3, 5, 6, and 8. Those contain the most distinctive SDK/object patterns.

14. Authoritative sources used and recommended

Primary sources were restricted to current Microsoft Learn / Microsoft documentation and MicrosoftLearning GitHub repositories, with the official exam study guide treated as the authority for scope.

1. Official AI-901 study guide (skills measured as of Apr 15, 2026) — https://learn.microsoft.com/en-us/credentials/certifications/resources/study-guides/ai-901

2. AI-901 exam page — https://learn.microsoft.com/en-us/credentials/certifications/exams/ai-901/

3. Get started with AI applications and agents on Azure learning path — https://learn.microsoft.com/en-us/training/paths/get-started-ai-apps-agents/

4. Generative AI and agents module — https://learn.microsoft.com/en-us/training/modules/get-started-with-generative-ai-and-agents/

5. Text analysis module — https://learn.microsoft.com/en-us/training/modules/get-started-text-analysis-azure/

6. Speech module — https://learn.microsoft.com/en-us/training/modules/get-started-speech-azure/

7. Computer vision module — https://learn.microsoft.com/en-us/training/modules/get-started-vision-azure/

8. Information extraction module — https://learn.microsoft.com/en-us/training/modules/get-started-information-extraction/

9. MicrosoftLearning AI fundamentals labs — https://github.com/MicrosoftLearning/mslearn-ai-fundamentals

10. AI-901 text-analysis lab — https://github.com/MicrosoftLearning/mslearn-ai-fundamentals/blob/main/Instructions/Exercises/03b-text-analysis.md

11. AI-901 speech lab — https://github.com/MicrosoftLearning/mslearn-ai-fundamentals/blob/main/Instructions/Exercises/04a-speech.md

12. Foundry SDK chat exercise (Responses API) — https://github.com/MicrosoftLearning/mslearn-ai-studio/blob/main/Instructions/Exercises/03-foundry-sdk.md

13. AIProjectClient Python reference — https://learn.microsoft.com/python/api/azure-ai-projects/azure.ai.projects.aiprojectclient

14. Azure Content Understanding documentation — https://learn.microsoft.com/en-us/azure/ai-services/content-understanding/

15. MicrosoftLearning information-extraction labs — https://github.com/MicrosoftLearning/mslearn-ai-information-extraction

Version note: Microsoft cloud services and SDK examples change quickly. This guide reflects the AI-901 blueprint and Microsoft Learn/GitHub content verified on 20 October 2026. Before the exam, re-check the official study guide for any newly announced skills-measured update.